

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «Избыточное кодирование данных в информационных системах. Код
Хемминга. Итеративные коды»

Выполнил:
Студент 4-го курса
Группы АС-66
Савинец М. Д.
Проверил:
Хацкевич А.С.

Цель работы: приобрести практические навыки кодирования и декодирования двоичных данных с использованием кода Хэмминга и итеративных кодов, исследовать обнаружение и исправление ошибок.

Ход работы

Вариант 11

Во второй части по циклическому распределению используется вариант 1 таблицы 5.2: 16 информационных бит, формы 4×4 , 8×2 , $4 \times 2 \times 2$ и $2 \times 4 \times 2$.

Часть 1 Код Хэмминга

Модель и реализация

Приложение написано на Python 3.10 или новее и использует только стандартную библиотеку. Информационные биты расположены первыми. Систематическая проверочная матрица $H = [P \mid I]$, кодовое слово $X_n = X_k \parallel X_r$, $X_r = P * X_k$, все операции по модулю 2. Синдром $S = H * Y_n$. Столбцы матрицы и синдром хранятся в виде битовых масок целых чисел Python. Ёмкость информационного блока ограничена 4096 битами; кодовое слово хранится как целое число.

Хэмминг: выбирается минимальное r с $2^r \geq k+r+1$. Информационные столбцы H ненулевые, попарно различные и не единичные; перебираются по увеличению веса и затем числового значения. Единичные столбцы зарезервированы для проверочных битов. При $k=16$: $(21,16)$, $d=3$; расширенный $(22,16)$, $d=4$.

Для $d=4$ добавленная строка в информационной части равна 1 XOR чётность старого столбца. Это результат преобразования общего паритета в канонический вид $[P \mid I]$. Все столбцы расширенного H имеют нечётный вес, а синдром двух ошибок - чётный: он не совпадёт с одиночным столбцом.

```
parity = 0
for i, column in enumerate(self.data):
    parity ^= column * ((message >> i) & 1)
return message | (parity << self.k)
```

Декодер сканирует все столбцы H . При единственном совпадении со S исправляет соответствующий бит, иначе оставляет слово и сообщает невозможность коррекции. Он не получает истинную кратность ошибки и не использует исходное сообщение. Работа с несколькими ошибками исследуется экспериментально; их полное исправление данным однократным декодером не обещается.

Файлы читаются как байты UTF-8, 8 бит на байт, старший первым, блоками до 512 байт. Поэтому длина всего файла не ограничена одним буфером. Это явное ограничение размера кодового блока, а не скрытое ограничение размера файла. Текст `examples/message.txt` содержит русский/английский алфавит, цифры и служебные символы.

Python использует динамическую память. Декодер просматривает все столбцы матрицы, но выполнение программы, операции над целыми числами и доступ к памяти не являются гарантированно `constant-time`. Требования общего README курса об отсутствии динамической памяти и `constant-time` этой Python-реализацией не выполняются.

Пример Хэмминга и матрицы

Сообщение 1011010010110100. Ниже приведены действительные выводы приложения для seed=11. Значения syndrome печатаются в порядке строк H. Изменение n меняет выбор случайных позиций, поэтому ошибки в примерах d=3 и d=4 разные.

```
k=16 r=5 n=21
H=[P|I] (row order: low syndrome bit first)
1101001000111011|10000
1010100100110110|01000
0110010010101101|00100
0001110001011100|00010
0000001111000011|00001
Xk=1011010010110100
Xr=01101
Xn=101101001011010001101
En_actual=00000001000000000100000
errors=2 seed=11
Yn=101101101011010101101
Yr=01101
Yr_recomputed=01001
S=00100
status=corrected_single_candidate
En_estimated=0000000000000000000100
Yn_corrected=101101101011010101001
data_decoded=1011011010110101
restored=0
```

```
k=16 r=6 n=22
H=[P|I] (row order: low syndrome bit first)
1101001000111011|100000
1010100100110110|010000
0110010010101101|001000
0001110001011100|000100
0000001111000011|000010
1111111111000000|000001
Xk=1011010010110100
Xr=011011
Xn=1011010010110100011011
En_actual=000000000010100000000000
errors=2 seed=11
Yn=1011010011100100011011
Yr=011011
Yr_recomputed=101000
S=110011
status=detected_uncorrectable
En_estimated=000000000000000000000000
Yn_corrected=1011010011100100011011
data_decoded=1011010011100100
restored=0
```

При двух ошибках обычный Хэмминг ошибочно корректирует третий бит. Расширенный код сообщает detected_uncorrectable и сохраняет полученное слово. Полный файловый протокол с матрицей H, Xk, Xr, Yn, Yr, S и векторами ошибок находится в results/hamming-file.txt; restored=1 подтверждает восстановление блока.

Часть 2 Итеративные коды

Итеративные схемы

Информационные биты индексируются (z,row,col). Для каждой плоскости вычисляются строчные и столбцовые проверки. В 2D третья группа содержит один паритет паритетов ($X_{hv} = \text{XOR}$ всех информационных бит). В 3D третья группа содержит диагонали row+col, четвёртая - row+cols-1-col, пятая - линии по z. Диагонали незамкнутые: rows+cols-1 на семейство в каждой плоскости, по обобщению рис. 5.3. Общий паритет в 3D дополнительно не включается. Этот точный выбор проверок нужно отличать от циклических диагоналей другой схемы 4x4x4 в рис. 5.4.

Каждая отдельная проверка получает свой проверочный символ и единичный столбец H. Количество групп - число геометрических семейств, а не число линейно независимых равенств только для данных. Зависимости между семействами данных не делают проверочную матрицу [P|I] вырожденной.

Код	k	r	n	R=k/n	r/n
H d=3	16	5	21	0.762	0.238
H d=4	16	6	22	0.727	0.273
4x4 g=2	16	8	24	0.667	0.333
4x4 g=3	16	9	25	0.640	0.360
8x2 g=2	16	10	26	0.615	0.385
8x2 g=3	16	11	27	0.593	0.407
4x2x2 g=2	16	12	28	0.571	0.429
4x2x2 g=3	16	22	38	0.421	0.579
4x2x2 g=4	16	32	48	0.333	0.667
4x2x2 g=5	16	40	56	0.286	0.714
2x4x2 g=2	16	12	28	0.571	0.429
2x4x2 g=3	16	22	38	0.421	0.579
2x4x2 g=4	16	32	48	0.333	0.667
2x4x2 g=5	16	40	56	0.286	0.714

Для фиксированных 16 бит форма 4x4 требует меньше строковых/столбцовых проверок, чем 8x2 (8 против 10). Минимальную избыточность даёт форма, близкая к квадрату; в 3D при осевых проверках - форма, близкая к кубу.

Пример матрицы 4x4 и одиночной ошибки (полная матрица H сохранена в results/iterative-example.txt).

```

layout: planes=1 rows=4 cols=4 groups=3
plane=0
1011
0100
1011
0100
k=16 r=9 n=25
Xk=1011010010110100
Xr=111100000
Xn=1011010010110100111100000
En_actual=0000000000000001000000000000
errors=1 seed=11
Yn=1011010010110000111100000
Yr=111100000
Yr_recomputed=111001001
S=000101001
status=corrected_single_candidate
En_estimated=0000000000000001000000000000
Yn_corrected=1011010010110100111100000
data_decoded=1011010010110100
restored=1

```

Методика эксперимента и результаты

Для каждой из 14 конфигураций выполнено 3000 испытаний каждой кратности 0–5: всего 252000 испытаний. Используется воспроизводимый SplitMix64 с seed=11, не криптографический генератор. Ровно i различных позиций выбираются частичной перестановкой без возвращения. Ошибки затрагивают и данные, и проверочные биты. Для линейного кода синдром ошибки не зависит от сообщения; в каждой конфигурации используется фиксированное псевдослучайное сообщение.

$N1$ - число испытаний; $D/N1$ - доля обнаруженных ошибок ($S \neq 0$); $C/N1$ - доля полностью восстановленных кодовых слов; $M/N1$ - ошибочные попытки коррекции. Полное восстановление означает равенство всех n бит, а не только данных. Идентификация позиции и кратности 1 проверяется полным перебором; для большей кратности совпадение синдрома с одним столбцом не является доказательством числа ошибок.

Код	i	$D/N1, \%$	$C/N1, \%$	$M/N1, \%$
H d=3	2	100.00	0.00	69.70
H d=3	3	96.33	0.00	62.47
H d=3	4	96.77	0.00	65.57
H d=4	2	100.00	0.00	0.00
H d=4	3	100.00	0.00	66.03
H d=4	4	96.50	0.00	0.00
4x4 g=2	2	100.00	0.00	17.73
4x4 g=2	3	99.43	0.00	16.00
4x4 g=2	4	98.87	0.00	10.20
4x4 g=3	2	100.00	0.00	0.00
4x4 g=3	3	100.00	0.00	16.67
4x4 g=3	4	99.20	0.00	0.00

Код	i	D/N1, %	C/N1, %	M/N1, %
8x2 g=2	2	100.00	0.00	13.87
8x2 g=2	3	99.30	0.00	14.77
8x2 g=2	4	99.53	0.00	6.37
8x2 g=3	2	100.00	0.00	0.00
8x2 g=3	3	100.00	0.00	16.27
8x2 g=3	4	99.50	0.00	0.00
4x2x2 g=2	2	100.00	0.00	13.40
4x2x2 g=2	3	99.67	0.00	5.87
4x2x2 g=2	4	99.67	0.00	3.07
4x2x2 g=3	2	100.00	0.00	0.00
4x2x2 g=3	3	100.00	0.00	0.53
4x2x2 g=3	4	100.00	0.00	0.00
4x2x2 g=4	2	100.00	0.00	0.00
4x2x2 g=4	3	100.00	0.00	0.00
4x2x2 g=4	4	100.00	0.00	0.00
4x2x2 g=5	2	100.00	0.00	0.00
4x2x2 g=5	3	100.00	0.00	0.00
4x2x2 g=5	4	100.00	0.00	0.00
2x4x2 g=2	2	100.00	0.00	12.97
2x4x2 g=2	3	99.63	0.00	5.00
2x4x2 g=2	4	99.90	0.00	2.57
2x4x2 g=3	2	100.00	0.00	0.00
2x4x2 g=3	3	100.00	0.00	0.70
2x4x2 g=3	4	99.97	0.00	0.00
2x4x2 g=4	2	100.00	0.00	0.00
2x4x2 g=4	3	100.00	0.00	0.00
2x4x2 g=4	4	100.00	0.00	0.07
2x4x2 g=5	2	100.00	0.00	0.00
2x4x2 g=5	3	100.00	0.00	0.00
2x4x2 g=5	4	100.00	0.00	0.00

Без ошибок все конфигурации возвращают исходное слово. Во всех испытаниях одиночные ошибки обнаружены и исправлены. Для $i \geq 2$ однобитный декодер не восстанавливает весь ошибочный вектор; увеличение групп главным образом снижает вероятность принять многобитную ошибку за одиночную. Значение 100% в случайной выборке не доказывает обнаружение всех ошибок данной кратности.

Сравнение и выводы

Рисунок 1. Доля ошибочных попыток коррекции при двух ошибках, % (3000 испытаний на конфигурацию).

Ошибочная коррекция двойных ошибок, %

H d=3		69.70%
H d=4		0.00%
4x4 g=2		17.73%
4x4 g=3		0.00%
8x2 g=2		13.87%
8x2 g=3		0.00%
4x2x2 g=2		13.40%
4x2x2 g=3		0.00%
4x2x2 g=4		0.00%
4x2x2 g=5		0.00%
2x4x2 g=2		12.97%
2x4x2 g=3		0.00%
2x4x2 g=4		0.00%
2x4x2 g=5		0.00%

Ошибочная коррекция двойных ошибок

Обычный Хэмминг наиболее экономичен, но синдром двойной ошибки часто совпадает с одиночным столбцом. Дополнительный общий паритет обеспечивает SECDED: одна ошибка исправляется, две обнаруживаются без неправильной коррекции. Итеративные коды проще интерпретировать геометрически; дополнительные семейства проверок повышают различимость ошибок ценой большего g и меньшей скорости R .

Четыре ошибки в вершинах прямоугольника информационной матрицы имеют нулевой синдром при проверках по строкам/столбцам; общий паритет их тоже не видит. Одиночная коррекция не превращает код в декодер произвольной кратности. По минимальному расстоянию код обнаруживает до $d-1$ ошибок в режиме контроля; гарантированно исправляет $\text{floor}((d-1)/2)$. В режиме одновременного исправления и обнаружения гарантии следует рассматривать отдельно (например, SECDED).

Результаты подтверждают корректность матриц, восстановление при одной ошибке, различие кодов Хэмминга $d=3/d=4$ и компромисс между избыточностью и достоверностью. Программа сохраняет ошибочные и исправленные слова, поэтому экспериментальные неудачи не скрываются.

Проверки, воспроизведение и источники

Успешно выполнены 12 автоматических тестов unittest на Python, внутри которых проверяются тысячи случаев. Протокол сохранён в results/tests.txt; сводка — в results/validation.json. Полный перебор 65536 сообщений для каждого кода Хэмминга $k=16$: $N \cdot X_n = 0$ вычисляется независимым умножением; минимальное расстояние найдено перебором и равно $3/4$. Перебраны все одиночные и двойные ошибки в тестируемых схемах; для SECDED проверен отказ от коррекции каждой

двойной ошибки. Проверены границы 4096 бит, UTF-8, много блоков файла, точная кратность ошибок и неверные параметры.

Среда измерений: Python 3.12.14, macOS, arm64. Метаданные запуска сохранены в results/environment.json.

```
python3 tools/verify.py
python3 lab2.py experiments 3000 11 1 --directory results
python3 lab2.py hamming-file examples/message.txt 4 1 11
python3 lab2.py iterative 1011010010110100 4 2 2 5 1 11
```

Артефакты: results/experiments.csv содержит все 84 строки результатов; results/hamming-file.txt - полный протокол файла; hamming-double-d3.txt и hamming-double-d4.txt - примеры двойных ошибок; iterative-example.txt - пример итеративного кода. Полный исходный код: lab/core.py, lab/experiments.py, lab2.py.

1. KiSB-2026-autumn, lab2:
<https://github.com/bstu-iit-information-security/KiSB-2026-autumn/tree/main/lab2>.
Материалы по коду Хэмминга: раздел 4.2; по итеративным кодам: раздел 5.2 и таблица 5.2.

2. Иванюк А. А., Мусин С. Б. Специальные главы высшей математики: теория помехоустойчивого кодирования. БГУИР, 2007. Дополнительные сведения о линейных кодах и синдроме: раздел 2.

3. Правила Fork/Branch/PR и безопасности: корневой README репозитория <https://github.com/bstu-iit-information-security/KiSB-2026-autumn>.